Project Description and ADRs

Course: 2110521 Software Architecture (2026/1)
Project: CampusKit

1. Project Name and Members

CampusKit is a peer-to-peer rental website for current Chulalongkorn University students. Students can list items they own, rent available items from other students, and arrange pickup and return on campus.

Name	**Student ID**
Chayutphong Soisri	6630054621
Thara Waranuset	6630132921
Thanyaporn Rattanacharoentham	6630138721
Pon Kittinaraporn	6630237121
Chalisa Wirojratana	6632045321

2. Project Description

2.1 Problem and Proposed Solution

Students may need equipment for only a short period, such as a camera for a course project, formal clothing for an interview, or a tent for a trip. Buying an item for occasional use can be costly, while other students may own suitable items that are currently unused.

Arranging rentals through social media and group chats can create several difficulties:

Problem	**How CampusKit addresses it**
Listings are scattered and hard to find across conversations.	A searchable catalogue brings rental offers together.
Owners must coordinate availability manually.	Date-based reservations prevent overlapping bookings.
Agreements and transaction status are unclear.	Both parties can view agreed terms, booking status, and rental history.
The item condition at pickup and return can be difficult to verify.	Pickup and return records preserve photographs and accessories; dispute adjudication is deferred.
Students have limited information about other participants.	Verified accounts and reviews provide a record of previous rentals.

These are the problems the project is designed to address, rather than findings from a completed user survey.

3. Target Customer and Actors

Actor	**Main responsibility**
Owner	List personally owned items, set availability, approve requests, and record handovers.
Renter	Find an item, request a rental, arrange payment, collect the item, and return it.
Platform Administrator	Review student verification and listing approval, and investigate failed payment or notification operations.
Payment Gateway	Production external actor for authorization, capture, release, refund, and payout. The current demo uses a local Payment Service ledger adapter without real money.
Email Provider	Production external actor for verification messages, booking updates, and reminders. External email/push delivery is deferred; the current demo uses an in-app inbox.

Owner and Renter are roles of the same Student account. A student may perform both roles across different transactions but cannot rent their own item. Authentication uses email/password login and a JWT; publishing and booking additionally require verified current student status. Email verification proves control of an address (the code is displayed on the UI in demo mode); an administrator reviews student status separately.
Scope for this iteration: implement authentication, listing, booking, demo payment, handover/return, reviews, and in-app notifications. Report handling, damage/loss claims, dispute adjudication, and a claim-window workflow are deferred. University IdP, real payment-provider, external email/push, and S3 integrations are also deferred. Photos use compressed inline data URLs in the demo. Administrator verification, listing approval, and technical recovery remain in scope.

4. Use Cases

UC-1: Publish an Equipment Listing

Field	**Description**
Primary actor	Owner
Supporting actors	Platform Administrator; Email Provider is a deferred production actor. The demo delivers notifications through the in-app inbox.
Goal	Make an item discoverable and bookable.
Trigger	The owner chooses to list an item.
Preconditions	The owner is signed in, verified, not suspended, and has a registered payout destination.
Success outcome	A published listing contains item details, prices, pickup options, and availability.

Main flow

1. The owner enters the item's name, category, description, condition, accessories, and replacement value.
2. The owner uploads photographs and sets the daily rate, deposit, and permitted rental duration.
3. The owner chooses campus pickup options and marks unavailable dates.
4. The system validates the information, saves a safe DRAFT, and requests calendar setup from Booking through gRPC. Publication waits until calendar setup succeeds.
5. The system screens the item. Flagged or high-value listings require administrator approval.
6. Once approved, the listing becomes searchable, and the owner receives a notification.

Alternative flows

- Incomplete or invalid information: reject the input and explain the validation error. A safe DRAFT is created only after validation succeeds, before calendar setup.

- Rejected listing: keep it unpublished and provide the reason. Editing its details and resubmitting it for review remain proposed behaviour; the current UI does not yet implement the complete workflow.
- Availability cannot be saved: do not publish until calendar setup succeeds.
- An existing listing is unpublished: stop new requests while preserving the terms of confirmed bookings. Cancelling those bookings follows the cancellation policy. Complete detail editing and resubmission remain outside the current demo.

UC-2: Book and Pay for a Rental

Field	**Description**
Primary actor	Renter
Supporting actors	Owner; Payment Gateway and Email Provider are deferred production actors. The demo uses a local payment ledger and in-app notifications.
Goal	Reserve an available item for an agreed period.
Trigger	The renter needs an item for selected dates.
Preconditions	The renter is signed in, verified, eligible to book, and has a supported payment method. The listing is published.
Success outcome	A confirmed booking has reserved dates, agreed terms, successful authorization, and a rental record awaiting pickup.

Main flow

1. The renter searches by keyword or category and filters by rental dates and campus location.
2. The system displays matching available listings, prices, and owner review summaries.
3. The renter selects pickup and return times. The system displays the rental fee, deposit, cancellation policy, and late-fee rule.
4. On submission, the system rechecks eligibility and availability and atomically reserves the requested period.
5. The owner approves or declines the request within the response window.

6. On approval, the system retains the reservation and requests separate authorizations for the fee and deposit.
7. After authorization succeeds, the system creates the rental record, confirms the booking, and notifies both parties.

Alternative flows

- Dates are no longer available: reject the request without authorizing payment.
- Owner declines or does not respond in time: close the request and release the dates.
- Authorization fails: permit retries within a 30-minute window, ending no later than pickup. If unsuccessful when the window expires, release the dates and reverse any successful partial authorization. The current demo does not enforce a one-retry limit.
- Confirmation fails after authorization: retry safely or cancel and reverse the authorization. Track unresolved reversals for administrator attention.
- Cancellation before pickup: update the rental if one exists, and release the dates.

UC-3: Complete the Handover, Return, and Settlement

Field	**Description**
Primary actors	Owner and Renter
Supporting actors	Platform Administrator for exceptions; Payment Gateway and Email Provider are deferred production actors. The demo uses a local payment ledger and in-app notifications.
Goal	Record pickup and return, then complete settlement.
Trigger	A confirmed rental reaches its pickup time.
Preconditions	A confirmed booking exists. The demo checks authorization readiness during fee capture at handover and during settlement; proactive advance payment-readiness checks and alerts remain proposed behaviour.
Success outcome	An accepted return is recorded, settlement succeeds, and the rental closes.

Main flow

1. The system checks payment readiness when processing handover and settlement. Advance checks and alerts before pickup remain proposed behaviour.
2. At the agreed location, both parties record pickup photographs, condition, and included accessories.
3. The renter confirms the owner's one-time pickup code. After rechecking authorization and successfully capturing the fee, the system records pickup. The deposit remains reserved.
4. The system tracks the due time and sends a return reminder.
5. At return, both parties first record return photographs, the item condition, and accessories. The owner then confirms the renter's one-time return code to accept the return.
6. After an accepted return, the system records the returned item as available. Items not yet returned remain blocked from new requests.
7. The system settles any late fee defined in the agreed booking terms, releases the remaining deposit, and pays the owner. There is no claim window or automated damage/loss deduction in this iteration.
8. After settlement succeeds, the rental closes and both parties may submit reviews.

Alternative flows

- Failed pickup: an invalid code or failed capture does not confirm handover. If the item is not as described, the renter may refuse it; cancel without a renter penalty and reverse payment.
- Late return: block new requests and identify affected future bookings. Notify those renters and offer an agreed reschedule or full cancellation without penalty. Keep the rental open until return is confirmed.
- Unaccepted return or missing item: keep the rental open and the item unavailable. Damage/loss claims and adjudication are outside the platform's current scope; the system does not determine compensation or deduct it automatically.
- Payment risk or settlement failure: an expired authorization blocks the affected capture or settlement operation and records an error. Preserve the custody record; leave an unsettled rental open and retry settlement explicitly after recovery. The demo ledger settles local financial changes atomically. Proactive deposit-expiry alerts, partial external-provider settlement recovery, and provider reconciliation remain production work.

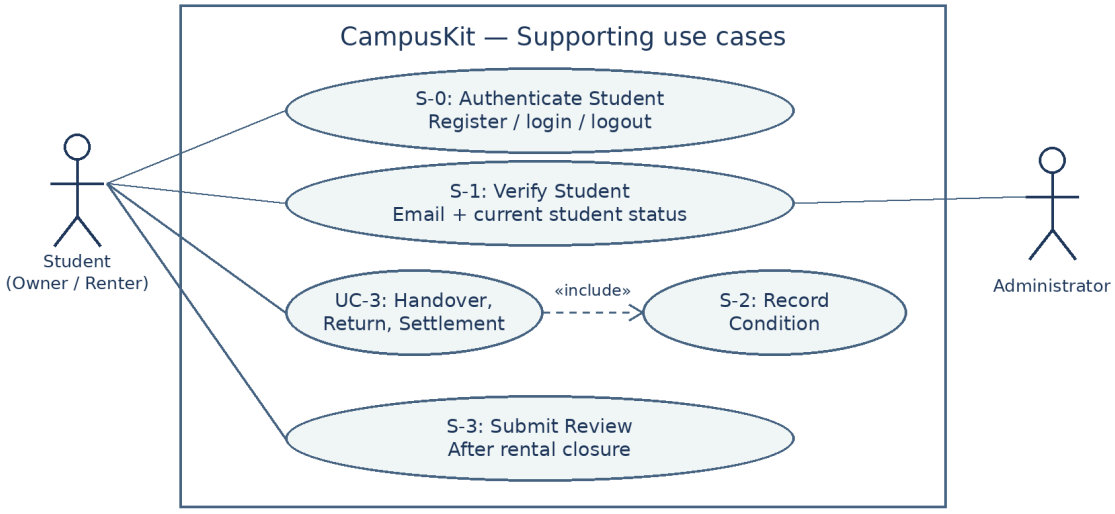
4.1 Supporting Use Cases

ID	Use case	Actors and behaviour
S-0	Authenticate Student	Student registers and logs in with email/password to receive a JWT; logout clears the client token. Authentication identifies the caller and is separate from email and current-student verification.
S-1	Verify Student	Student verifies control of their email address (the code is displayed on the UI in demo mode); an administrator separately reviews current student status. Both checks precede publishing and booking and are distinct from S-0 authentication.
S-2	Record Condition	Owner and renter record pickup and return evidence. Included in UC-3.
S-3	Submit Review	Either participant reviews the other after rental closure. Closure is a precondition, not an extension relationship.

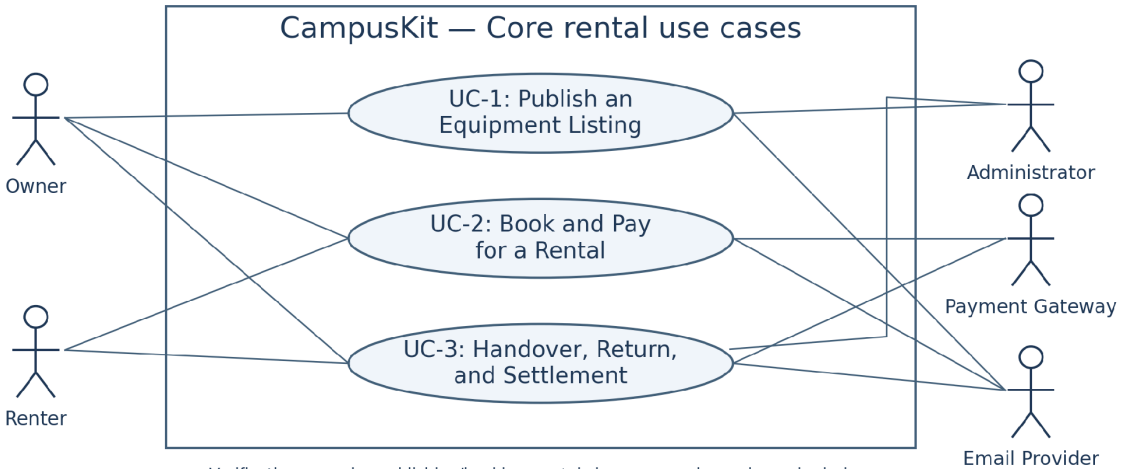
4.2 Use Case Diagram

The two diagrams show the core rental flow and supporting use cases within the current CampusKit scope. Actors are outside the system boundary; ellipses are use cases. Solid lines indicate participation. S-0 Authenticate Student covers registration, login, and logout; S-1 Verify Student separately covers email and current-student verification. Authentication and verification are preconditions for publishing and booking, not mandatory included steps

on every request. UC-3 includes S-2 Record Condition. Rental closure is a precondition for S-3 Submit Review. Payment Gateway and Email Provider in the core diagram represent deferred production actors; the demo substitutes the local payment ledger and in-app inbox.



Authentication and student verification precede publishing / booking.
Rental closure is a precondition for review submission.



Verification precedes publishing/booking; rental closure precedes review submission.

5. Key Requirements

5.1 Functional Requirements

ID	Requirement	Use cases
FR-01	Authenticate students with JWTs, verify email and current student status, and enforce role, resource ownership, and rental-participant permissions at protected services.	S-0, S-1
FR-02	Manage listings, photographs, prices, moderation, and availability.	UC-1
FR-03	Search by category, keyword, dates, and campus location; show review summaries.	UC-2
FR-04	Generate and preserve quotes and atomically reserve dates; support approval, rejection, expiry, and cancellation.	UC-2
FR-05	Track fee and deposit authorization, capture, release, refund, and payout through a payment adapter.	UC-2, UC-3
FR-06	Record pickup and return through one-time codes and condition evidence.	UC-3, S-2

ID	Requirement	Use cases
FR-07	Detect overdue rentals, calculate late fees, block unavailable items, and handle affected bookings.	UC-3
FR-08	Support participant reviews, booking/rental history, and itemised payment status.	UC-2, UC-3, S-3
FR-09	Deliver updates and reminders through PostgreSQL transactional outbox events, RabbitMQ, and the Notification Service's MongoDB inbox. Track recoverable workflow errors; external email/push delivery is deferred.	UC-1, UC-2, UC-3

5.2 Non-Functional Requirements

These are acceptance targets, not measured results. Planned performance testing will use 300 concurrent virtual users, 1,500 listings, a documented request mix, and the payment test adapter.

ID	Requirement and verification
NFR-01	Under the stated test load, p95 search latency is at most 1 second and single-resource reads at most 500 ms. Server errors remain below 0.5%; expected business rejections are counted separately. Verify by load testing.
NFR-02	Concurrent requests cannot create overlapping reservations. Retried operations cannot duplicate a capture, deduction, refund, or payout. Verify with concurrency and timeout-after-success tests.

ID	Requirement and verification
NFR-03	Production target: external access uses HTTPS and internal calls use TLS. The local Docker demo uses HTTP and insecure gRPC. Private evidence and transactions require authorised access; raw card data is not stored or logged. Verify permissions and logs locally; verify TLS before production deployment.
NFR-04	Each service owns its database and supports independent deployment through compatible API contracts. Verify database permissions and consumer contract tests.
NFR-05	A notification outage does not reverse a successful rental operation. Failed deliveries and payment steps remain recoverable and traceable with correlation IDs. Verify by simulated outages and recovery.
NFR-06	Acceptance target: the three core use cases work on a 360-pixel-wide viewport without horizontal scrolling, with labelled forms and keyboard access. At least four of five users with preverified test accounts can find an item and submit a request within 10 minutes. Responsive layout checks and the five-user usability study are separate validations; the study has not yet been completed.

ID	Requirement and verification
NFR-07	Acceptance targets: RPO 24 hours and RTO 4 hours for PostgreSQL and MongoDB. Verify backups and recovery through a restoration exercise; persistent Docker volumes alone do not demonstrate these targets. A completed backup/restore drill and real-provider payment reconciliation after recovery remain outstanding.

The demo includes Thai/English labels and uses THB; complete bilingual localization remains deferred. Before operation with real users, validate identity verification, payment-provider capabilities, TLS, privacy notices, and data retention rules. These are deployment dependencies rather than completed integrations.

6. Architecture Decision Records

The eight records below follow the Nygard structure: Context, Decision, Status, and Consequences. Each record states the alternatives considered, the selected approach, and its trade-offs. All eight decisions are Accepted; acceptance records a design choice, not proof that every production capability has been implemented. Payment-provider selection and the production deposit strategy remain separate, deferred decisions.

ADR-001: Decompose by Business Capability

Context: CampusKit has a five-person team and one term to deliver independent services. A modular monolith offers simpler deployment and local transactions; microservices make independent deployment and failure recovery explicit but require more contracts and operations. Use service decomposition as the course architecture, with business responsibilities listed here as boundaries rather than as separate ADRs.

Decision: Use seven independently deployable services: Student Identity, Listing, Booking, Rental, Payment, Review and Reputation, and Notification, behind one API gateway. Student Identity owns verification and account standing. Each service exposes APIs and owns its database; no service reads another service's tables. Report handling and dispute adjudication are deferred and have no service in this iteration.

Status: Accepted.

Consequences: Separate deployment and data ownership support the course's architecture goals. The team must assign owners to seven services, maintain API contracts and a common local startup configuration, and test cross-service failures with correlated logs. This accepts more operational work than a modular monolith; reducing the feature scope is necessary to keep the course implementation manageable.

ADR-002: Authenticate Centrally, Authorise at the Data Owner

Context: Gateway-only authorisation centralises checks but cannot reliably evaluate current resource ownership without duplicating service data. Checking every permission at each service keeps decisions next to the protected resource but adds repeated checks and internal identity configuration. A valid login alone does not establish listing ownership or rental participation.

Decision: Student Identity authenticates email/password login and issues JWTs. The gateway and protected services validate JWTs; the gateway propagates the token to REST services and as metadata on gRPC calls. Services ignore client-supplied identity headers and user IDs as evidence of permission. The data-owning service checks the caller's role,

ownership, and rental participation. Service-to-service operations require internal service authentication and endpoint-specific permissions; internal calls do not bypass authorisation. Local HTTP and insecure gRPC are restricted to the Docker demo network; TLS is the production transport target.

Status: Accepted.

Consequences: Resource permissions remain with the owning service. Configure JWT validation, trusted token propagation, internal service credentials, and production TLS; test forged identities, missing/invalid tokens, unauthorised direct calls, and access to another student's records. Email verification and administrator review of student status remain separate eligibility checks. Every new protected resource requires explicit authorisation.

ADR-003: Prevent Overlapping Reservations with Database Constraints

Context: Owner blackouts, pending requests, and confirmed bookings compete for one item. An application-level check followed by an insert can race under concurrent requests. Serialising requests with a distributed lock adds lease and recovery complexity. A PostgreSQL exclusion constraint enforces non-overlap at the storage boundary but requires range modelling and conflict handling.

Decision: Keep the authoritative calendar in Booking. Represent blocking periods as half-open intervals (start, end), and enforce non-overlap for the same item with a PostgreSQL GiST exclusion constraint on item identity and the time range (using btree_gist for scalar identity). Store owner blackouts and active reservations in the constrained calendar. Create or change a reservation within one transaction; convert a constraint conflict into an unavailable-dates response. Expiry or cancellation explicitly removes the blocking interval in a transaction. Listing publication waits for calendar setup, and search availability remains advisory.

Status: Accepted.

Consequences: The database prevents overlapping committed periods even when requests arrive together. Back-to-back rentals are permitted by half-open intervals. The team must test simultaneous overlapping inserts, adjacent intervals, owner blackout changes, and expiry-versus-confirmation races. An expiry worker releases stale reservations; a delayed worker can temporarily reduce availability but must not allow double booking. Booking remains a dependency for publication and reservation.

ADR-004: Database per Service with Relational and Document Stores

Context: Shared tables simplify joins but couple service schemas. Reservations, custody, and financial records need transactions and relational constraints; reviews and notification inbox entries fit document storage. PostgreSQL and MySQL are viable relational options; retain PostgreSQL for Booking's range/exclusion constraints and the existing relational setup. MongoDB provides document storage for reviews and inbox records. The course implementation requires both an RDBMS and a NoSQL database; using two technologies adds operational work compared with the original all-PostgreSQL baseline.

Decision: Use PostgreSQL for Student Identity (identity_db), Listing (listing_db), Booking (booking_db), Rental (rental_db), and Payment (payment_db). Use MongoDB for Review and Reputation (review_db) and Notification (notification_db). Each service owns a separate logical database and credentials. The local deployment shares one PostgreSQL instance and one MongoDB instance. Cross-service access uses published APIs rather than another service's tables or collections.

Status: Accepted.

Consequences: Each service controls its schema. PostgreSQL enforces reservation constraints and unique financial operation identifiers; MongoDB compound unique indexes prevent duplicate reviews and inbox records. Maintain per-service migrations/index setup, credentials, backups, and recovery tests for both technologies. Separate logical databases preserve ownership, while shared instances remain infrastructure failure points.
Cross-service reads require APIs and failed workflows require recovery logic.

ADR-005: Orchestrate Workflows with Idempotent Recovery

Context: Booking confirmation and settlement cross service boundaries. A timeout does not prove failure, and a local rollback cannot undo a completed remote payment. Event choreography would distribute the sequence across services; a dedicated orchestrator would add another deployment. Keeping orchestration with the service responsible for the outcome makes the initial workflows easier to locate and test.

Decision: Booking orchestrates confirmation and cancellation; Rental orchestrates handover and settlement. Persist workflow state and stable financial operation identifiers; Payment enforces uniqueness and returns recorded results for completed duplicates. Compensate failed confirmation by reversing successful authorizations and releasing the reservation. After an accepted return, persist SETTLEMENT_PENDING, release the calendar, attempt settlement, and close only after success. Failed settlement remains available for explicit retry. The demo Payment ledger performs local financial changes atomically; real-provider uncertain outcomes, partial settlement, and reconciliation remain production work. Domain

transactions write notification events to a transactional outbox for asynchronous delivery through RabbitMQ (ADR-008); broker failure does not undo committed business state.

Status: Accepted.

Consequences: Booking and Rental gain workflow and compensation logic. Test duplicate requests, timeouts after remote success, failed confirmation, settlement retry, and broker outage/recovery. Notification delivery may lag without reversing a successful rental operation. The local demo does not demonstrate partial external-provider settlement or a complete administrator reconciliation queue; these require further implementation and operational procedures before production.

ADR-006: Isolate Payment Integration Behind an Adapter

Context: Marketplace payouts and authorization durations depend on the provider. Direct provider calls throughout the application would couple business rules to one SDK. An adapter requires extra mapping code but permits sandbox and production implementations behind the same contract. Selecting an unverified renewal interval would make deposit protection uncertain.

Decision: Payment owns its adapter and transaction ledger. The current demo uses a local ledger adapter without real money to model separate fee/deposit authorizations, fee capture at pickup, and settlement after accepted return. Model authorization expiry explicitly; the deposit remains reserved after fee capture until settlement. Local settlement changes are atomic and financial operation identifiers are unique. Real-provider integration, provider reconciliation, provider selection, and the production deposit strategy remain deferred.

Status: Accepted.

Consequences: The adapter keeps provider-specific code out of Booking and Rental. Test local ledger outcomes, expiry, declined authorization, duplicate requests, and settlement retry. Before production, validate authorization duration, renewal and payouts, then implement uncertain/partial provider-outcome reconciliation and proactive payment-risk alerts that preserve custody and distinguish secured funds from shortfalls. The demo is not evidence that these external-provider capabilities are complete.

ADR-007: Frontend Framework

Context: The project needs responsive student and administrator web interfaces for listings, booking, handover, and payment status. Compare React with TypeScript and Vue 3 with TypeScript on component structure, setup effort, and learning cost. Both support TypeScript; using one language across the stack does not distinguish them.

Dimension	React with TypeScript	Vue 3 with TypeScript
Component structure	JSX/TSX keeps rendering logic close to TypeScript; reusable typed components and hooks.	Single-file components group templates, logic, and styles; reusable logic through the Composition API.
Advantages for this project	Retains the proposed React baseline and supports shared listing cards, booking forms, and status views.	Provides an explicit template structure and component organisation suited to the same interfaces.
Costs and learning	Requires agreed routing/data-fetching tools, plus JSX, hooks, and state conventions.	Requires learning template directives, reactivity, and .vue files; switching would change the proposed frontend conventions.
TypeScript support	Supported; a shared language is useful but does not distinguish the two choices.	Supported; a shared language is useful but does not distinguish the two choices.

Selection rationale: retain the proposed React baseline to avoid changing direction within one term, and accept the extra tooling decisions. Vue remains a viable alternative. This is a project-specific continuity trade-off; it does not assume greater React experience or a universal performance advantage.

Decision: Use React with TypeScript for the student and administrator web interfaces. Keep one shared frontend project with reusable components and a documented routing, API-access, and form-handling convention. Backend framework selection is outside this frontend ADR.

Status: Accepted.

Consequences: Reusing components and typed API models can keep the web interfaces consistent. The team must document dependency versions, build/test commands, routing

and data-fetching choices, and loading/error states. Members unfamiliar with React need onboarding time, and JSX/hooks require agreed review conventions. Validate NFR-06 with keyboard access, labelled forms, and a 360-pixel viewport. The frontend consumes published APIs and does not contain authoritative permission, availability, or settlement rules.

ADR-008: RabbitMQ Events with Transactional Outbox and Idempotent Inbox

Context: Notifications must not make successful rental operations fail when delivery is unavailable. Direct synchronous notification calls couple business transactions to delivery. Publishing independently of database commits can lose events or announce uncommitted changes. A transactional outbox adds publisher/consumer work but keeps events consistent with domain state.

Decision: Identity, Listing, Booking, and Rental write outbox events in the same PostgreSQL transaction as the relevant domain state. A publisher sends persistent events to a durable RabbitMQ exchange and marks the outbox entry sent only after publisher confirmation. Notification consumes a durable queue and upserts MongoDB inbox documents before acknowledging delivery. A unique (eventId, recipient) index deduplicates redelivery. Malformed events go to a dead-letter queue; transient storage failures are requeued. The current delivery channel is the in-app inbox; external email/push remains deferred.

Status: Accepted.

Consequences: Broker delivery is at least once; the unique inbox index ensures one record per event and recipient. Broker/storage outages may delay notifications, which resume after recovery without reversing committed business operations. Test broker outage, publisher confirmation, consumer acknowledgement, and duplicate deliveries. Queue retention, bounded retry/backoff, dead-letter recovery, broker high availability, and external delivery operations require further operational work.